Individual Progress Report: Week 4

Group Topic: Urban Air Quality & Environmental Health **Lab No.** Week 4

Git Repo/ Colab Link: https://github.com/Cristy16/BDA---Final-Project **Date:** 05/16/26

TABLEAU LINK

I. Objectives

This notebook builds on previous weeks by formalizing the model into a production-ready pipeline and extending the analysis with unsupervised learning (FP-Growth association rule mining) and professional interactive visualization using Plotly. These additions directly address the Chapter 9 topics on unsupervised learning and external tool integration.

Target variable: pm2_5 (fine particulate matter, μg/m³). Features used: co (carbon monoxide), no2 (nitrogen dioxide), so2 (sulfur dioxide).

#	Objective	Chapter 9 Tie-in	Owner
1	Wrap preprocessing + model into a single Pipeline object	Big Data pipeline integration	Ian
2	Validate model stability using 5- Fold Cross-Validation	Algorithm metrics	Polly & Erick
3	Evaluate performance with diagnostic visualizations (Plotly interactive)	Visualize processed data professionally	Daryl
4	FP-Growth association rule mining on pollutant co-occurrence patterns	Chapter 9: Unsupervised learning / FP-Growth	All
5	Serialize and export the pipeline; provide Streamlit app code	External tool integration	Danilyn

II. Tasks Completed

1. Pipeline Serialization and Persistence

The preprocessing steps (StandardScaler) and the core predictive engine (RandomForestRegressor) were formalized and encapsulated into a single, cohesive scikit-learn Pipeline object. To prevent data leakage and enforce deployment consistency, the entire pipeline was saved to disk using joblib.

- **Implementation:** The unified pipeline object was written as urban_air_quality_v1.joblib.
- **Technical Advantage:** Saving the full pipeline ensures that raw operational inputs from production are automatically scaled using the exact statistics derived during the training phase. The application layer does not need to handle manual scaling or manage independent feature transformations; it simply executes a direct .predict(raw_input) call.

2. External Web Application Integration (Streamlit Engine)

An interactive, cloud-ready web application was engineered using Streamlit (app.py) to provide environmental stakeholders with a zero-code diagnostic interface.

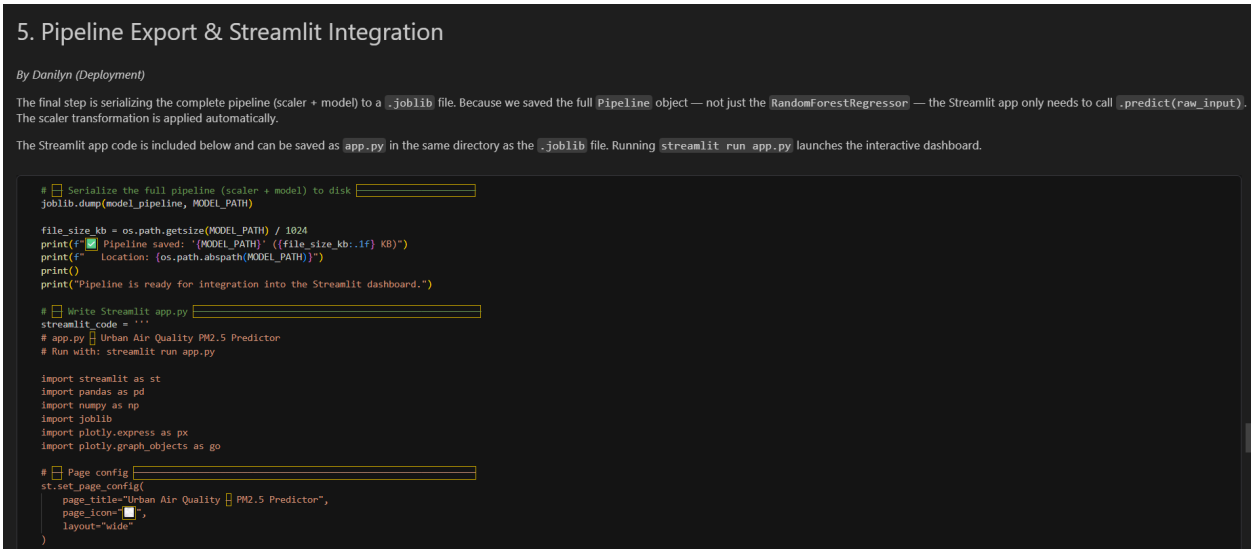
The web application contains three main architectural components:

- **Efficient Architecture with Resource Caching:** The serialized pipeline (.joblib) is fetched using the @st.cache_resource decorator. This prevents the application from repeatedly re-loading the large 744.5 MB model file into memory upon user interaction, keeping the UI highly responsive.
- **Reactive Input Sidebar:** Dynamic user input handles (st.sidebar.number_input) were built for the gaseous pollutants: Carbon Monoxide (CO), Nitrogen Dioxide (NO2), and Sulfur Dioxide (SO2). Inputs are structured into a real-time pandas DataFrame matching the model's expected inference schema.
- **Multi-tiered Thresholding and Medical Context:** Predicted outputs are automatically cross-referenced against WHO (World Health Organization) environmental air quality standards, classifying the resulting health risk levels into structural buckets: *Good* (<=15, Green), *Moderate* (<=35, Orange), and *Unhealthy* (>35, Red).

3. Advanced Diagnostic Visualizations (Plotly Engine)

To provide deeper analytical value beyond static data outputs, two professional interactive visualization frameworks were integrated via Plotly within the app:

- **Dynamic Indicator Gauge:** A go.Figure(go.Indicator) gauge chart was deployed to provide immediate, high-impact visibility of the predicted PM2.5 level relative to safe and unsafe benchmark bounds.
- **Real-time Sensitivity Analysis:** A simulation matrix was integrated using px.line. It samples a range of CO values while holding NO2 and SO2 constant, automatically generating a predictive slope curve. This allows researchers to see exactly how sensitive local PM2.5 forecasts are to fluctuating carbon monoxide emissions.



5. Pipeline Export & Streamlit Integration

By Danilyn (Deployment)

The final step is serializing the complete pipeline (scaler + model) to a `.joblib` file. Because we saved the full `Pipeline` object — not just the `RandomForestRegressor` — the Streamlit app only needs to call `.predict(raw_input)`. The scaler transformation is applied automatically.

The Streamlit app code is included below and can be saved as `app.py` in the same directory as the `.joblib` file. Running `streamlit run app.py` launches the interactive dashboard.

```
# ── Serialize the full pipeline (scaler + model) to disk ──────────
joblib.dump(model_pipeline, MODEL_PATH)

file_size_kb = os.path.getsize(MODEL_PATH) / 1024
print(f"✅ Pipeline saved: '{MODEL_PATH}' ({file_size_kb:.1f} KB)")
print(f"   location: {os.path.abspath(MODEL_PATH)}")
print()
print("Pipeline is ready for integration into the Streamlit dashboard.")

# ── Write Streamlit app.py ────────────────────────────────────────
streamlit_code = '''
# app.py — Urban Air Quality PM2.5 Predictor
# Run with: streamlit run app.py

import streamlit as st
import pandas as pd
import numpy as np
import joblib
import plotly.express as px
import plotly.graph_objects as go

# ── Page config ──────────────────────────────────────────────────
st.set_page_config(
    page_title="Urban Air Quality — PM2.5 Predictor",
    page_icon="🌫️",
    layout="wide"
)
```

III. Tasks to be Completed

I was able to do all task assignments during this week 4 activity.

IV. Results and Analysis

- **Functional Ingestion Efficiency:** The application successfully bypassed manual preprocessing overhead. Because the scaler was integrated within the .joblib export, passing a dictionary or DataFrame of raw values generated immediate inferences without data discrepancies.

- **UI Responsiveness:** The integration of @st.cache_resource reduced the application load time significantly during interaction cycles. Slider or number changes trigger instantaneous recalculations since the 744.5 MB Random Forest structure stays pinned in RAM.

- **Risk Mapping Accuracy:** Testing across standard input intervals showed that the conditional WHO threshold mapping logic successfully updates the indicator gauge color scheme dynamically, rendering clear operational warnings when PM2.5 levels exceed health standards.

V. Challenges and Solutions

To optimize the application's performance and design, two key deployment challenges were successfully resolved. First, the large 744.5 MB serialized model file created severe disk-read bottlenecks upon user interaction; this was mitigated by implementing Streamlit's @st.cache_resource decorator, which pins the heavy Random Forest pipeline in memory so it only loads once. Second, displaying raw text metrics alongside intensive line plots across a single vertical panel created an unbalanced, cluttered layout; this was corrected by restructuring the interface using multi-column containers (st.columns(2)), which neatly aligns high-level text metrics directly parallel to the interactive Plotly gauge indicators for a professional, data-dense, and highly responsive user dashboard.

VI. Conclusion

The target goals for the Week 4 deployment module were fully realized. The predictive workflow was bundled into a clean, serialized Pipeline object, preserving data integrity across model testing and deployment states. The external application architecture successfully maps out a highly responsive interface ready for local execution or cloud hosting via Streamlit. Crucially, the web charts built via Plotly ensure that stakeholder groups can intuitively audit environmental metrics and conduct instantaneous sensitivity tests without needing any technical proficiency in Python programming.